Chapter 8

Path Testing

Outline

- Preliminaries
- Program graphs
- DD-Paths
- Test coverage metrics
- Basis path testing
- Essential complexity (from McCabe)

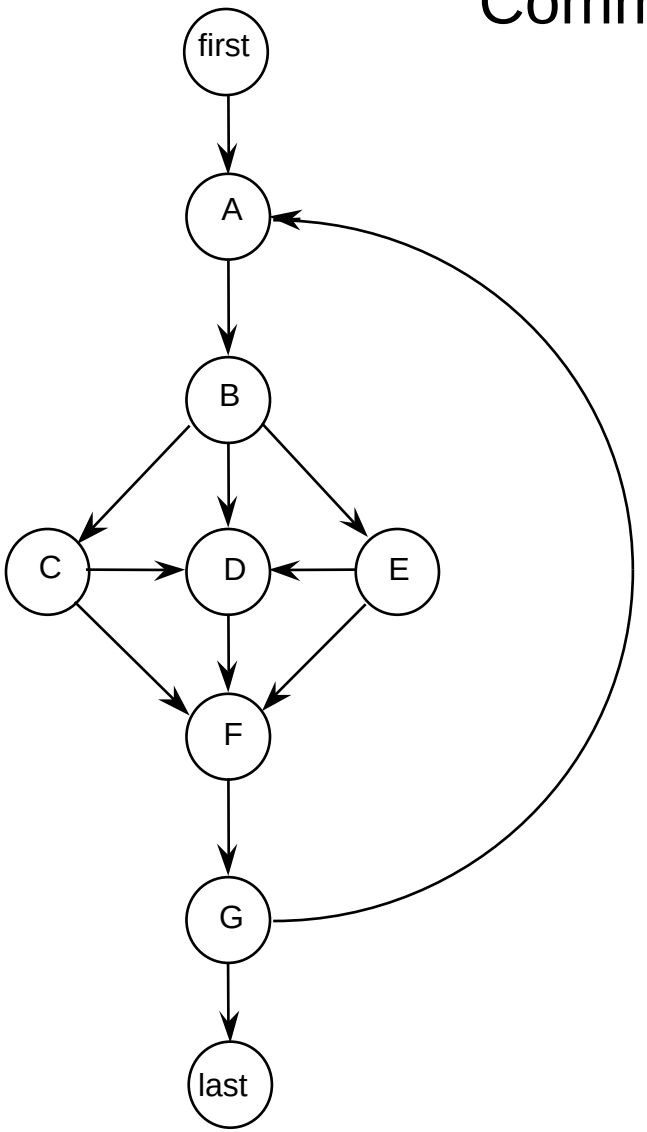
Code-Based (Structural) Testing

- Complement of/to Specification-based (Functional) Testing
- Based on Implementation
- Powerful mathematical formulation
  - program graph
  - define-use path
  - Program slices
- Basis for Coverage Metrics (a better answer for gaps and redundancies)
- Usually done at the unit level
- Not very helpful to identify test cases
- Extensive commercial tool support

Path Testing

- Paths derived from some graph construct.
- When a test case executes, it traverses a path.
- Huge number of paths implies some simplification is needed.
- Big Problem: infeasible paths.
- Big Question: what kinds of faults are associated with what kinds of paths?
- By itself, path testing can lead to a false sense of security.

Common Objection to Path-Based Testing (Trillions of Paths)

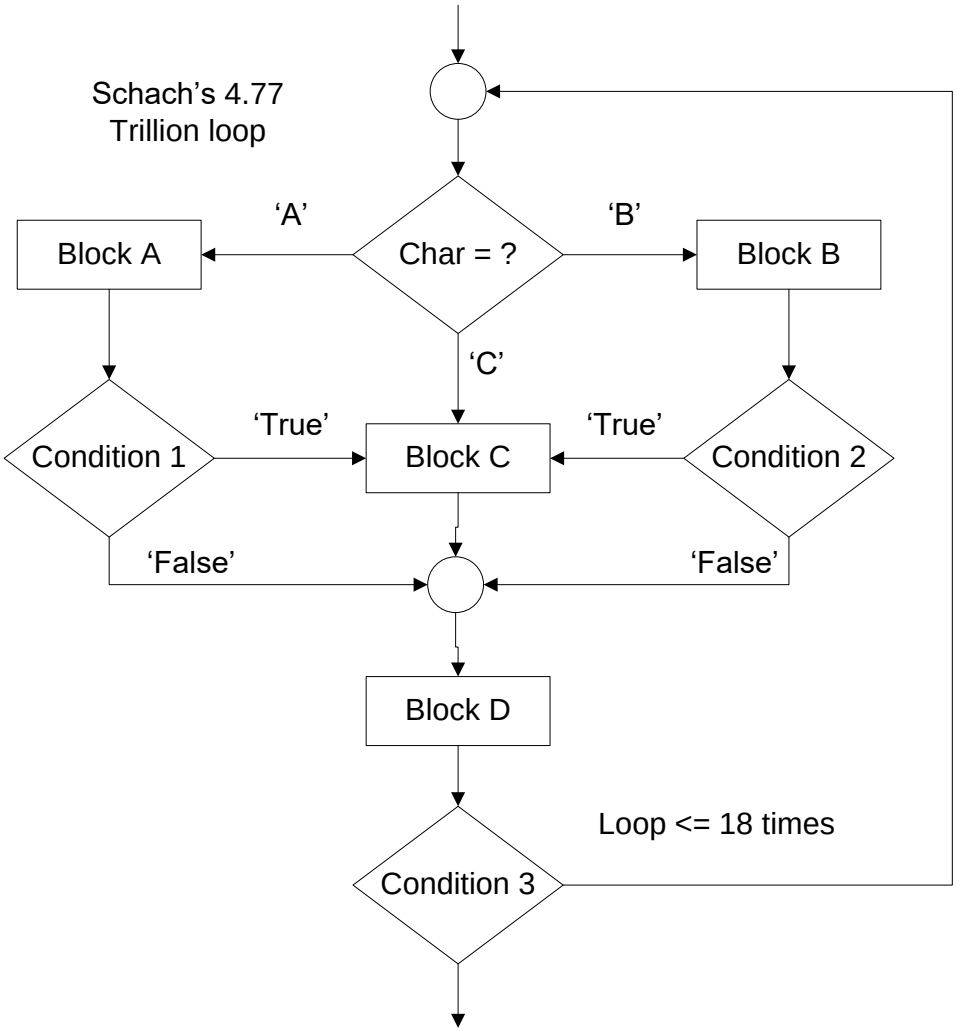


If the loop executes up to 18 times, there are 4.77 Trillion paths. Impossible, or at least, infeasible, to test them all. [Schach]

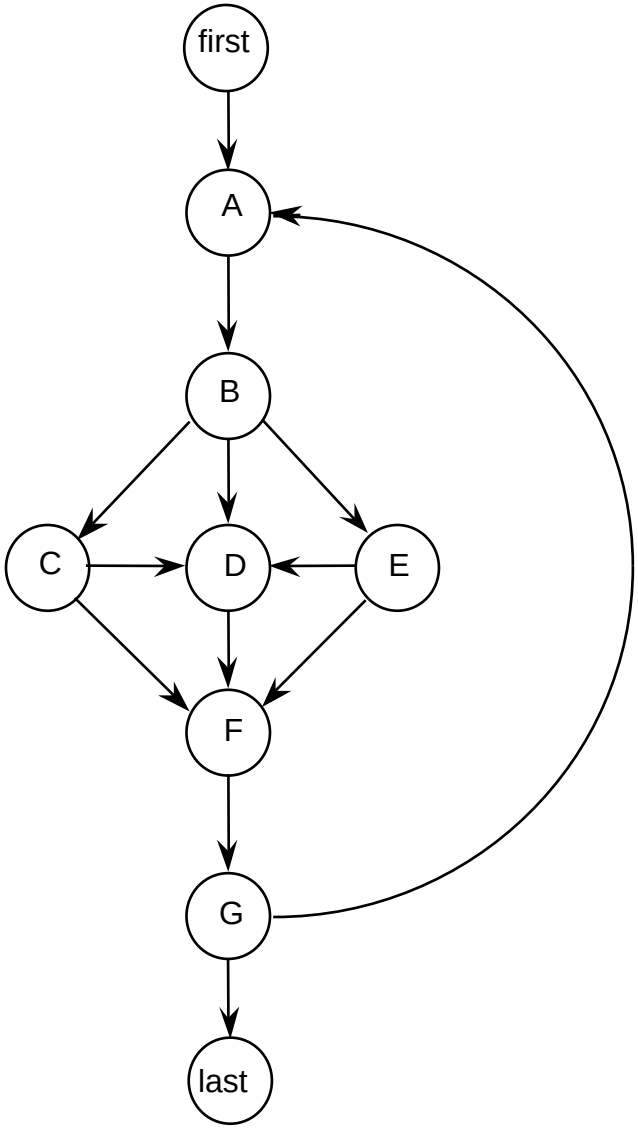
$5^0 + 5^1 + 5^2 + \ldots + 5^{18} = 4{,}768{,}371{,}582{,}030$

Stephen R. Schach, *Software Engineering,* , (2nd edition) Richard D. Irwin, Inc. and Aksen Associates, Inc. 1993

Schach's flowchart



Test Cases for Schach's "Program"



1. First-A-B-C-F-G-Last
2. First-A-B-C-D-F-G-Last
3. First-A-B-D-F-G-A-B-D-F-G-Last
4. First-A-B-E-F-G-Last
5. First-A-B-E-D-F-G-Last

These test cases cover

- Every node
- Every edge
- Normal repeat of the loop
- Exiting the loop

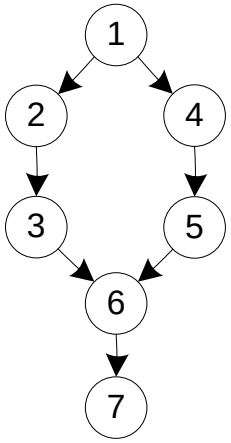
Program Graphs

Definition: Given a program written in an imperative programming language, its *program graph* is a directed graph in which nodes are statement fragments, and edges represent flow of control. (A complete statement is a “default” statement fragment.)

Program Graphs of Structured Programming Constructs

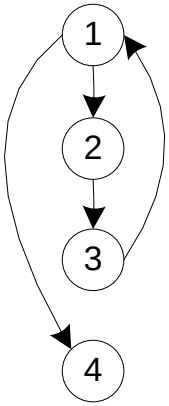
If-Then-Else

```
1   If <condition>
2      Then
3         <then statements>
4      Else
5         <else statements>
6   End If
7   <next statement>
```



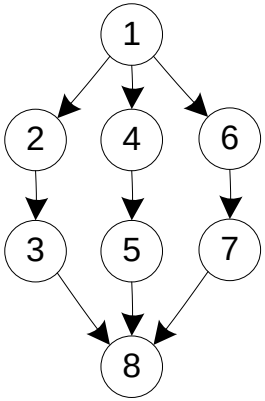
Pre-test Loop

```
1   While <condition>
2      <repeated body>
3   End While
4   <next statement>
```



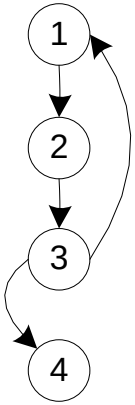
Case/Switch

```
1   Case  n Of 3
2      n=1:
3         <case 1  statements>
4      n=2:
5         <case 2  statements>
6      n=3:
7         <case 3  statements>
8   End Case
```



Post-test Loop

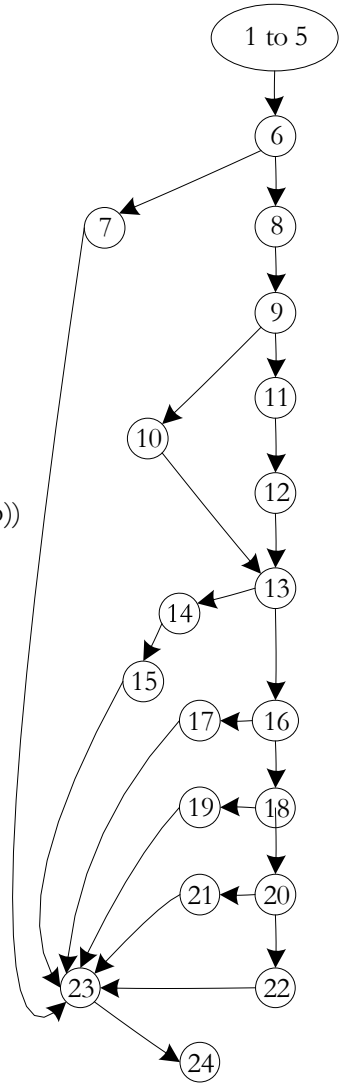
```
1   Do
2      <repeated body>
3   Until <condition>
4   <next statement>
```



Sample Program Graph

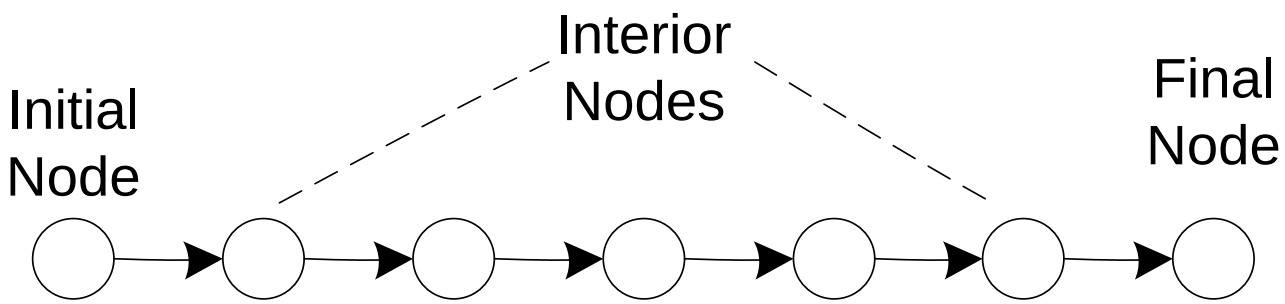
(Figure 8.2 redrawn)

```
1   public static int triangle3(int a, int b, int c) {
1
1     boolean c1, c2, c3, isATriangle;
1
1     // Step 1: Validate Input
2       c1 = (1 <= a) && (a <= 300);
3       c2 = (1 <= b) && (b <= 300);
4       c3 = (1 <= c) && (c <= 300);
5
5       int triangleType = INVALID;
6       if(!c1 || !c2 || !c3)
7         triangleType = OUT_OF_RANGE;
8       else {
8
8       // Step 2: Is A Triangle?
9         if((a < b + c) && (b < a + c) && (c < a + b))
10            isATriangle = true;
11          else
12            isATriangle = false;
12
12        // Step 3: Determine Triangle Type
13          if(isATriangle) {
14            if((a == b) && (b == c))
15              triangleType = EQUILATERAL;
16            else if((a != b) && (a != c) && (b != c))
17              triangleType = SCALENE;
18            else
19              triangleType = ISOSELES;
20          } else
21            triangleType = INVALID;
22      }
23
23      return triangleType;
24    }
```



DD-Paths

- Originally defined by E. F. Miller (1977?)
- “DD” is short for “decision to decision”
- Original definition was for early (second generation) programming languages
- Similar to a “chain” in a directed graph
- Basis of interesting test coverage metrics



More Definitions

Definition

A ***chain*** in a directed graph is a sequence of nodes in which the initial node has an indegree >= 0, and the final node has an outdegree >= 0. Every internal node has indegree = outdegree = 1. In a degenerate case, a chain could consist of a single node

Definition

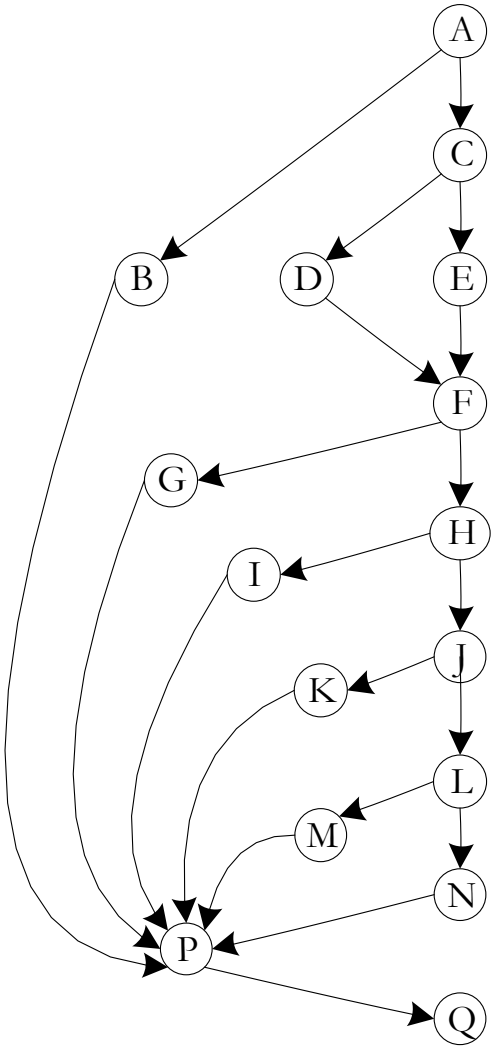
A ***DD-Path*** is a chain of statement fragments.

Definition

Given a program written in an imperative language, its ***DD-Path graph*** is the directed graph in which nodes are DD-Paths of its program graph, and edges represent control flow between successor DD-Paths.

DD-Path Graph of Figure 8.2

Figure 8.2	*DD-Path*
1 - 6	A
7	B
8, 9	C
10	D
11, 12	E
13	F
14, 15	G
16	H
17	I
18	J
19	K
20	L
21	M
22	N
23	P
24	Q



Exercises and Questions

- Compute the cyclomatic complexity of
  - the Triangle problem program graph
  - the Triangle problem DD-Path graph
- Are the complexities equal?
- What conclusions can you draw?
- Recall the discussion of Condensation Graphs from Chapter 4. Can you make a statement about 2-connected components in a Program Graph?

Code-Based Test Coverage Metrics

- Used to evaluate a given set of test cases
- Often required by
  - contract
  - U.S. Department of Defense
  - company-specific standards
- Elegant way to deal with the gaps and redundancies that are unavoidable with specification-based test cases.
- BUT
  - coverage at some defined level may be misleading
  - coverage tools are needed

Code-Based Test Coverage Metrics

(E. F. Miller, 1977 dissertation)

- C_0: Every statement
- C_1: Every DD-Path
- C_{1p}: Every predicate outcome
- C_2: C_1 coverage + loop coverage
- C_d: C_1 coverage +every pair of dependent DD-Paths
- C_{MCC}: Multiple condition coverage
- C_{ik}: Every program path that contains up to k repetitions of a loop (usually k = 2)
- C_{stat}: "Statistically significant" fraction of paths
- C_∞: All possible execution paths

Test Coverage Metrics from Program Graphs

- Every node
- Every edge
- Every chain
- Every path

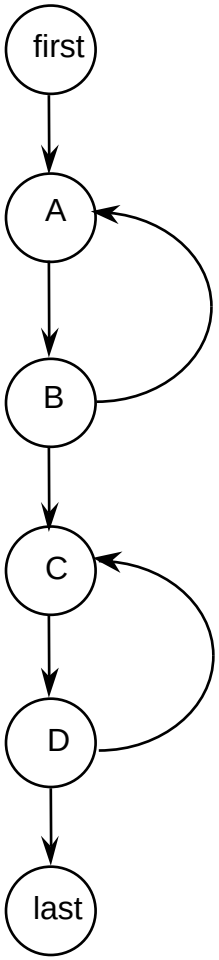
- How do these compare with Miller's coverage metrics?

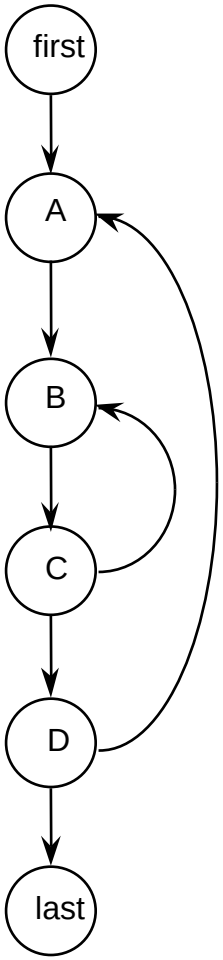
Testing Loops

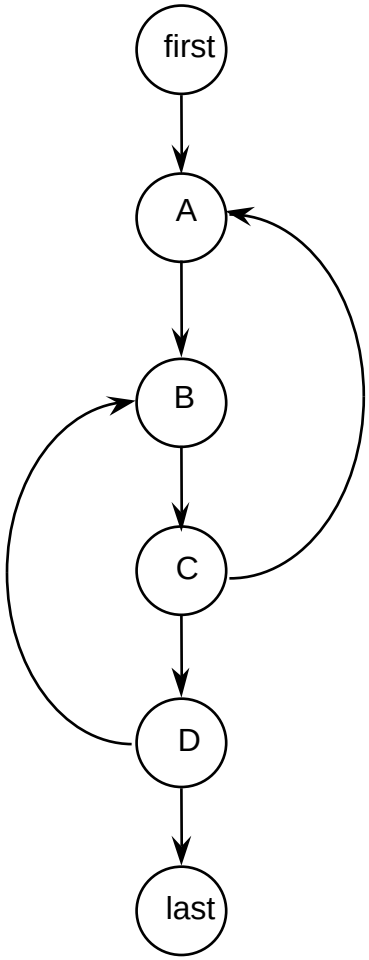
Huang's Theorem: (Paraphrased)
Everything interesting will happen in two loop traversals: the normal loop traversal and the exit from the loop.

Exercise:
Discuss Huang's Theorem in terms of graph based coverage metrics.

Concatenated, Nested, and Knotted Loops







Strategy for Loop Testing

- Huang's theorem suggests/assures 2 tests per loop is sufficient. (Judgment required, based on reality of the code.)
- For nested loops:
  - Test innermost loop first
  - Then "condense" the loop into a single node (as in condensation graph, see Chapter 4)
  - Work from innermost to outermost loop
- For concatenated loops: use Huang's Theorem
- For knotted loops: Rewrite! (see McCabe's cyclomatic and essential complexity)

Multiple Condition Testing

- Consider the multiple condition as a logical proposition, i.e., some logical expression of simple conditions.
- Make the truth table of the logical expression.
- Convert the truth table to a decision table.
- Develop test cases for each rule of the decision table (except the impossible rules, if any).
- Next 3 slides: multiple condition testing for

```
If (a < b + c) AND (b < a + c) AND (c < a + b)
    Then IsATriangle = True
    Else  IsATriangle = False
Endif
```

Truth Table for Triangle Inequality

(a<b+c) AND (b<a+c) AND (c<a+b)

(a<b+c)	(b<a+c)	(c<a+b)	(a<b+c) AND (b<a+c) AND (c<a+b)
T	T	T	T
T	T	F	F
T	F	T	F
T	F	F	F
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	F

Decision Table for

(a<b+c) AND (b<a+c) AND (c<a+b)

c1: a<b+c	T	T	T	T	F	F	F	F
c2: b<a+c	T	T	F	F	T	T	F	F
c3: c<a+b	T	F	T	F	T	F	T	F
a1: impossible				X		X	X	X
a2:Valid test case #	1	2	3		4			

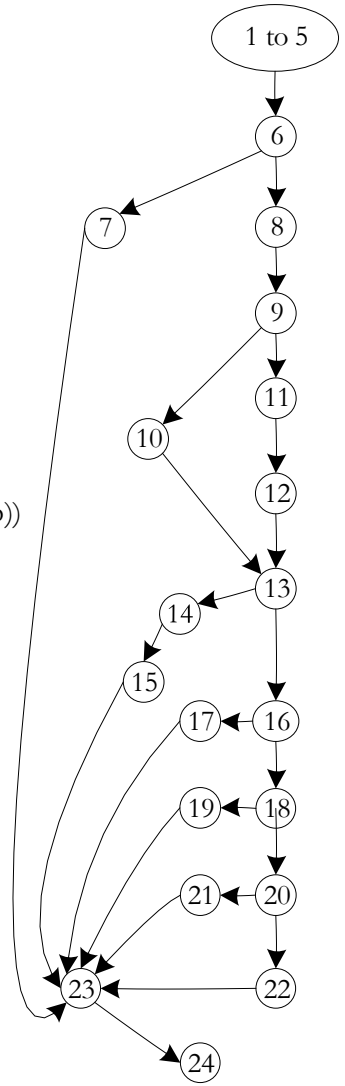
Multiple Condition Test Cases for

(a<b+c) AND (b<a+c) AND (c<a+b)

Test Case		a	b	c	expected output
1	all true	3	4	5	TRUE
2	c ≥ a + b	3	4	9	FALSE
3	b ≥ a + c	3	9	4	FALSE
4	a ≥ b + c	9	3	4	FALSE

Note: could add test cases for c = a + b, b = a + c, and a = b + c.

```
1   public static int triangle3(int a, int b, int c) {
1
1     boolean c1, c2, c3, isATriangle;
1
1     // Step 1: Validate Input
2       c1 = (1 <= a) && (a <= 300);
3       c2 = (1 <= b) && (b <= 300);
4       c3 = (1 <= c) && (c <= 300);
5
5       int triangleType = INVALID;
6       if(!c1 || !c2 || !c3)
7         triangleType = OUT_OF_RANGE;
8       else {
8
8       // Step 2: Is A Triangle?
9         if((a < b + c) && (b < a + c) && (c < a + b))
10          isATriangle = true;
11        else
12          isATriangle = false;
12
12      // Step 3: Determine Triangle Type
13        if(isATriangle) {
14          if((a == b) && (b == c))
15            triangleType = EQUILATERAL;
16          else if((a != b) && (a != c) && (b != c))
17            triangleType = SCALENE;
18          else
19            triangleType = ISOSELES;
20        } else
21          triangleType = INVALID;
22      }
23
23      return triangleType;
24    }
```



Dependent DD-Paths

(Figure 8.2 redrawn)

Dependent DD-Paths can correspond to feasible and infeasible paths.

Feasible Paths in Figure 8.2

- If a path traverses node 10, it must traverse nodes 14 – 19
- If a path traverses node 12, it must traverse nodes 20 and 21

Infeasible Paths in Figure 8.2

- If a path traverses node 10, it cannot traverse nodes 20 and 21
- If a path traverses node 12, it cannot traverse nodes 14 - 19

Test Coverage for Compound Conditions

- Extension of/to Multiple Condition Testing
- Modified Condition Decision Coverage (MCDC)
- Required for “Level A” software by DO-178B
- Three variations*
  - Masking MCDC
  - Unique-Cause MCDC
  - Unique-Cause + Masking MCDC
- Masking MCDC is
  - the weakest of the three, AND
  - is recommended for DO-178B compliance

* Chilenski, John Joseph, "An Investigation of Three Forms of the Modified Condition Decision Coverage (MCDC) Criterion," DOT/FAA/AR-01/18, April 2001.

[http://www.faa.gov/about/office_org/headquarters_offices/ang/offices/tc/library/]

Chilenski's Definitions

- Conditions can be either *simple* or *compound*
  - isATriangle is a simple condition
  - (a < b + c) AND (b < a + c) is a compound condition
- Conditions are *strongly coupled* if changing one always changes the other.
  - (x = 0) and (x ≠ 0) are strongly coupled in

    ((x = 0) AND a) OR ((x ≠ 0) AND b)
- Conditions are *weakly coupled* if changing one may change one but not all of the others.
  - All three conditions are weakly coupled in

  ((x = 1) OR (x = 2) OR (x = 3))
- *"Masking"* is based on the Domination Laws
  - (x AND false)
  - (x OR true)

MCDC requires...

- Every statement must be executed at least once,
- Every program entry point and exit point must be invoked at least once,
- All possible outcomes of every control statement are taken at least once,
- Every non-constant Boolean expression has been evaluated to both True and False outcomes,
- Every non-constant condition in a Boolean expression has been evaluated to both True and False outcomes, and
- Every non-constant condition in a Boolean expression has been shown to independently affect the outcomes (of the expression).

MCDC Variations

- "*Unique-Cause MCDC* [requires] a unique cause (toggle a single condition and change the expression result) for all possible (uncoupled) conditions."

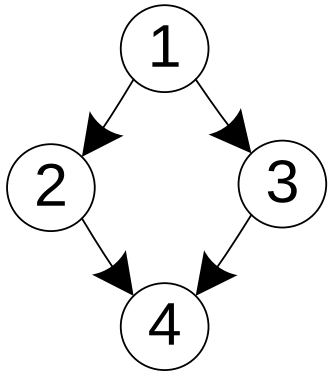
- "*Unique-Cause + Masking MCDC* [requires] a unique cause (toggle a single condition and change the expression result) for all possible (uncoupled) conditions. In the case of strongly coupled conditions, masking [is allowed} for that condition only, i.e., all other (uncoupled) conditions will remain fixed."

MCDC Variations (continued)

- "*Masking MCDC* allows masking for all conditions, coupled and uncoupled. (toggle a single condition and change the expression result) for all possible (uncoupled) conditions. In the case of strongly coupled conditions, masking [is allowed} for that condition only (i.e., all other (uncoupled) conditions will remain fixed."

Example

1. If (a AND (b OR c))
2. Then y = 1
3. Else y = 2
4. EndIf



Decision Table for (a AND (b OR c))

Conditions	**rule 1**	**rule 2**	**rule 3**	**rule 4**	**rule 5**	**rule 6**	**rule 7**	**rule 8**
a	T	T	T	T	F	F	F	F
b	T	T	F	F	T	T	F	F
c	T	F	T	F	T	F	T	F
a AND (b OR c)	True	True	True	False	False	False	False	False
Actions								
y = 1	x	x	x	—	—	—	—	—
y = 2	—	—	—	x	x	x	x	x

For MCDC Coverage of (a AND (b OR c))

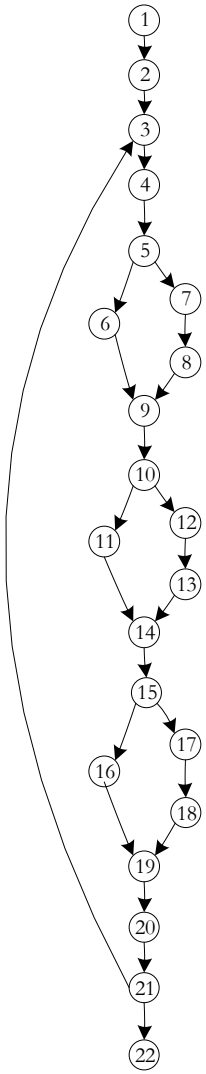
- Rules 1 and 5 toggle condition a
- Rules 2 and 4 toggle condition b
- Rules 3 and 4 toggle condition c

- If we expand (a AND (b OR c)) to

  ((a AND b) OR (a AND C)), we cannot do unique cause testing on variable a because it appears in both sub-expressions.

A NextDate Example

```
1.    int day, int month, int year;
1.    boolean dayOK, monthOK, yearOK, rangesOK;
1.  java.util.Scanner inputScanner = new java.util.Scanner(System.in);

2.  rangesOK = False;

3.  Do
3.     {
4.        month = inputScanner.nextInt();
4.        day = inputScanner.nextInt();
4.        year = inputScanner.nextInt();
4.        inputScanner.close();

5.        if ((day > 0) &&(day < 32))
6.           dayOK = True;
7.        else {
8.           dayOK = False;
9.        }

10.       if ((month > 0) &&(month < 13))
11.          monthOK = True;
12.       else {
13.          monthOK = False;
14.       }

15.       if ((year > 1841) &&(year < 2043))
16.          yearOK = True;
17.       else {
18.          yearOK = False;
19.       }
20.    }

21.  Until (dayOK && monthOK && yearOK);

22.  rangesOK = True;
```



dayOK&&monthOK&&yearOK

Decision Table for dayOK&&monthOK&&yearOK

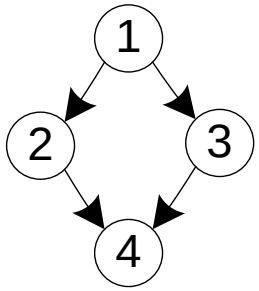
Conditions	**rule 1**	**rule 2**	**rule 3**	**rule 4**	**rule 5**	**rule 6**	**rule 7**	**rule 8**
dayOK	T	T	T	T	F	F	F	F
monthOK	T	T	F	F	T	T	F	F
YearOK	T	F	T	F	T	F	T	F
The Until condition	True	False	False	False	False	False	False	False
Actions								
Leave the loop	x	—	—	—	—	—	—	—
Repeat the loop	—	x	x	x	x	x	x	x

Test Cases and Coverage for dayOK && monthOK && yearOK

- Decision Coverage: Rule 1 and any of Rules 2 – 8
- Multiple Condition Coverage: all eight rules are needed
- Modified Condition Decision Coverage:
  rules 1 and 2 toggle yearOK
  rules 1 and 3 toggle monthOK
  rules 1 and 4 toggle dayOK

One more example

1. If (a < b + c) AND (a < b + c) AND (a < b + c)
2. Then IsA Triangle = True
3. Else IsA Triangle = False
4. EndIf



In the three conditions, there are interesting dependencies that create four impossible rules.

Corresponding Decision Table

Conditions	**rule 1**	**rule 2**	**rule 3**	**rule 4**	**rule 5**	**rule 6**	**rule 7**	**rule 8**
(a < b + c)	T	T	T	T	F	F	F	F
(b < a + c)	T	T	F	F	T	T	F	F
(c < a + b)	T	F	T	F	T	F	T	F
IsATriangle = True	x	—	—	—	—	—	—	—
IsATriangle = False	—	x	x	—	x	—	—	—
impossible	—	—	—	x		x	x	x

Two Strategies of MCDC Testing

- Rewrite the code as a decision table
  - algebraically simplify
  - watch for impossible rules
  - eliminate masking when possible
- Rewrite the code as nested If logic
  - (see example of the Triangle Program fragment on the next slide)

Test Cases and Coverage

- Decision Coverage: Rule 1 and Rule 2. (also, rules 1 and 3, or rules 1 and 5.
- Rules 4, 6, 7, and 8 are impossible.
- Condition Coverage
  - rules 1 and 2 toggle (c < a + b)
  - rules 1 and 3 toggle (b < a + c)
  - rules 1 and 5 toggle (a < b + c)
- Modified Condition Decision Coverage:
  - rules 1 and 2 toggle (c < a + b)
  - rules 1 and 3 toggle (b < a + c)
  - rules 1 and 5 toggle (a < b + c)

Code-Based Testing Strategy

- Start with a set of test cases generated by an “appropriate” (depends on the nature of the program) specification-based test method.
- Look at code to determine appropriate test coverage metric.
  - Loops?
  - Compound conditions?
  - Dependencies?
- If appropriate coverage is attained, fine.
- Otherwise, add test cases to attain the intended test coverage.

Test Coverage Tools

- Commercial test coverage tools use “instrumented” source code.
  - New code added to the code being tested
  - Designed to “observe” a level of test coverage
- When a set of test cases is run on the instrumented code, the designed test coverage is ascertained.
- Strictly speaking, running test cases in instrumented code is not sufficient
  - Safety critical applications require tests to be run on actual (delivered, non-instrumented) code.
  - Usually addressed by mandated testing standards.

Capabilities of Selected Code Coverage Tools

Tool name	Line/ Statement	Block	Untouched code	Decision	Condition	MCDC	Path
Clover	x	x	x	x			
JaCoCo	x	x	x				
McCabeIQ	x	x	x	x	x	x	x
Parasoft Jtest	x	x	x	x	x	x	x
squish coco	x	x	x	x	x		
Testwell	x	x	x	x	x	x	

Basis Path Testing

- Proposed by Thomas McCabe in 1982
- Math background
  - a Vector Space has a set of independent vectors called basis vectors
  - every element in a vector space can be expressed as a linear combination of the basis vectors
- Example: Euclidean 3-space has three basis vectors
  - (1, 0, 0) in the x direction
  - (0, 1, 0) in the y direction
  - (0, 0, 1) in the z direction
- The Hope: If a program graph can be thought of as a vector space, there should be a set of basis vectors. Testing them tests many other paths.

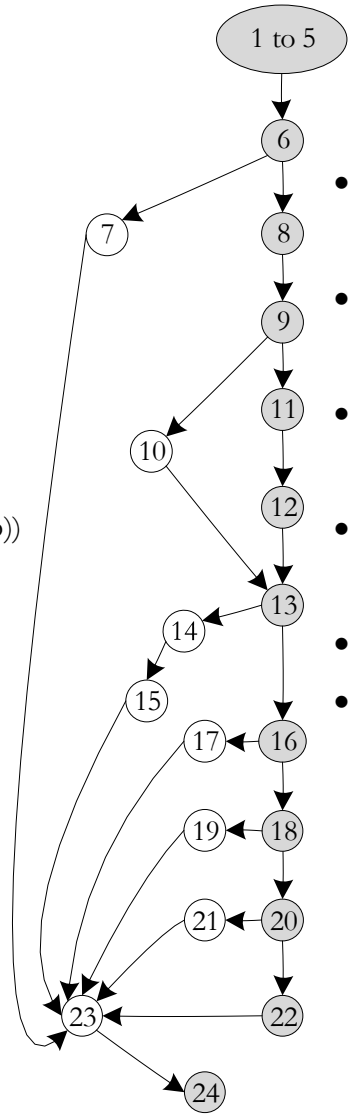
(McCabe) Basis Path Testing

- in math, a basis "spans" an entire space, such that everything in the space can be derived from the basis elements.
- the cyclomatic number of a strongly connected directed graph is the number of linearly independent cycles.
- given a program graph, we can always add an edge from the sink node to the source node to create a strongly connected graph. (assuming single entry, single exit)
- computing V(G) = e - n + p from the modified program graph yields the number of independent paths that must be tested.
- since all other program execution paths are linear combinations of the basis path, it is necessary to test the basis paths. (Some say this is sufficient; but that is problematic.)
- the next few slides follow McCabe's Basis Path technique in the Triangle Program (Figure 8.2, repeated next).

Basis Path Procedure

1. Select a baseline path, which should correspond to some "normal case" program execution.
2. Choose a path with as many decision nodes as possible.
3. Retrace the baseline path in turn each decision is "flipped"; that is, when a node of outdegree ≥ 2 is reached, a different edge must be taken.
4. Each new path is clearly independent of the previous path.
5. The flipping process continues until we have the cyclomatic number of paths

```
1   public static int triangle3(int a, int b, int c) {
1
1     boolean c1, c2, c3, isATriangle;
1
1     // Step 1: Validate Input
2       c1 = (1 <= a) && (a <= 300);
3       c2 = (1 <= b) && (b <= 300);
4       c3 = (1 <= c) && (c <= 300);
5
5       int triangleType = INVALID;
6       if(!c1 || !c2 || !c3)
7         triangleType = OUT_OF_RANGE;
8       else {
8
8       // Step 2: Is A Triangle?
9         if((a < b + c) && (b < a + c) && (c < a + b))
10          isATriangle = true;
11        else
12          isATriangle = false;
12
12      // Step 3: Determine Triangle Type
13        if(isATriangle) {
14          if((a == b) && (b == c))
15            triangleType = EQUILATERAL;
16          else if((a != b) && (a != c) && (b != c))
17            triangleType = SCALENE;
18          else
19            triangleType = ISOSELES;
20        } else
21          triangleType = INVALID;
22      }
23
23      return triangleType;
24    }
```



Finding Baseline Paths

(Figure 8.2 redrawn)

- The shaded nodes are the first baseline path.
- "Flips" occur at nodes 9, 13, 16, 18, 20,
- The cyclomatic complexity of this program graph is 7
- Counting the first path, we only have 6 independent paths.
- Go back and flip node 6.
- The results of this procedure are in Table 8.6, repeated on the next slide.

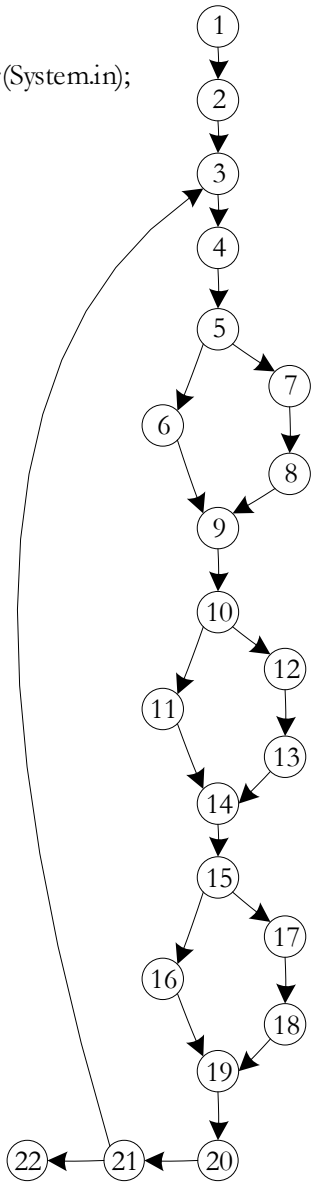
Result of Basis Paths Procedure

Path	Node Sequence	triangleType
p1	1-6, 8, 9, 11, 12, 13, 16, 18, 20, 22, 23, 24	(infeasible path)
p2 flip 9	1-6, 8, 9, 10, 13, 16, 18, 20, 22, 23, 24	(infeasible path)
p3 flip 13	1-6, 8, 9, 11, 12, 13, 14, 15, 23, 24	EQUILATERAL
p4 flip 16	1-6, 8, 9, 11, 12, 13, 16, 17, 23, 24	SCALENE
p5 flip 18	1-6, 8, 9, 11, 12, 13, 16, 18, 19, 23, 24	ISOSCELES
p6 flip 20	1-6, 8, 9, 11, 12, 13, 16, 18, 20, 21, 23, 24	NOT A TRIANGLE
p7 flip 6	1-6, 7, 23, 24	OUT OF RANGE

Another Basis Path Example

(Figure 8.8 redrawn)

```
1.    int day, int month, int year;
1.    boolean dayOK, monthOK, yearOK, rangesOK;
1.  java.util.Scanner inputScanner = new java.util.Scanner(System.in);

2.  rangesOK = False;

3.  Do
3.     {
4.        month = inputScanner.nextInt();
4.        day = inputScanner.nextInt();
4.        year = inputScanner.nextInt();
4.        inputScanner.close();

5.        if ((day > 0) &&(day < 32))
6.           dayOK = True;
7.        else {
8.           dayOK = False;
9.        }

10.        if ((month > 0) &&(month < 13))
11.           monthOK = True;
12.        else {
13.           monthOK = False;
14.        }

15.        if ((year > 1841) &&(year < 2043))
16.           yearOK = True;
17.        else {
18.           yearOK = False;
19.        }
20.    }

21.  while ( !(dayOK && monthOK && yearOK));

22.   rangesOK = True;
```



- The Baseline path takes the four True branches.
- "Flips" occur at nodes 5, 10, 15, *21,*
- *The cyclomatic complexity of this* program graph is 5
- The results of this procedure are in Table 8.7, repeated on the next slide.
- The Basis Path Method works pretty well for this example.

Result of Basis Paths Procedure

Path	Node Sequence	Result
p1	1, 2, 3, 4, 5, 6, 9, 10, 11, 14, 15, 16, 19, 20, 21	rangesOK = True
p2 flip 5	1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 14, 15, 16, 19, 20, 21, 3	dayOK = False
p3 flip 10	1, 2, 3, 4, 5, 6, 9, 10, 12, 13, 14, 15, 16, 19, 20, 21, 3	monthOK = False
p4 flip 15	1, 2, 3, 4, 5, 6, 9, 10, 11, 14, 15, 17, 18, 19, 20, 21, 3	yearOK = False
p5 flip 21	1, 2, 3, 4, 5, 6, 9, 10, 11, 14, 15, 16, 19, 20, 21, 22	Exit loop

Observations on the Basis Path Method

Pros

1. The identified basis paths are independent.
2. The correct (cyclomatic number) of paths are identified.

Cons

1. Infeasible paths can be identified (but they clearly can never be tested.)
2. Even if every basis path is feasible, they might not constitute an adequate test.
3. The set of paths in a program graph does is not a vector space (see next slide).

Postscript (for mathematicians only!)

For a set V to be a vector space, two operations (addition and scalar multiplication) must be defined for elements in the set. In addition, the following criteria must hold for all vectors x, y, and z ϵ V, and for all scalars k, l, 0, and 1:

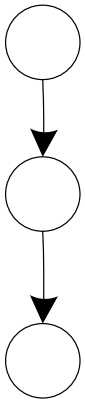
a. if x, y ϵ V, the vector x + y ϵ V.
b. x + y = y + x.
c. (x + y) + z = x + (y + z).
d. there is a vector 0 ϵ V such that x + 0 = x.
e. for any x ϵ V, there is a vector -x ϵ V such that x + (-x) = 0.
f. for any x ϵ V, the vector kx ϵ V, where k is a scalar constant.
g. k(x + y) = kx + ky.
h. (k + l)x = kx + lx.
i. k(lx) = (kl)x.
j. 1x = x.

“Structured Programming”

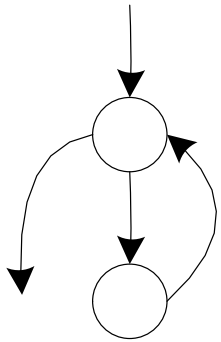
- “Structured Programming” is a coding discipline that was developed in the 1970s as a response to “spaghetti code”.
- It is sometimes known as “GoTo-less” programming, as the Go To statement is not needed.
- One result of Structured Programming is that the resulting program graphs are “clean” and usually have lower cyclomatic complexity.
- The precepts of Structured Programming can (should!) be applied to object-oriented code.

Program Graphs of the "Structured Programming" Constructs

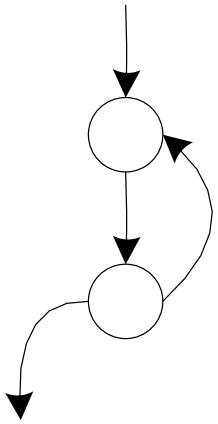
Sequence



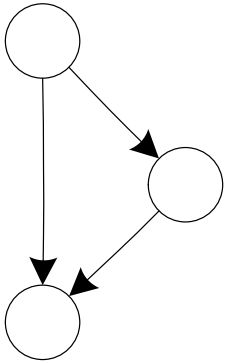
Pre-test Loop



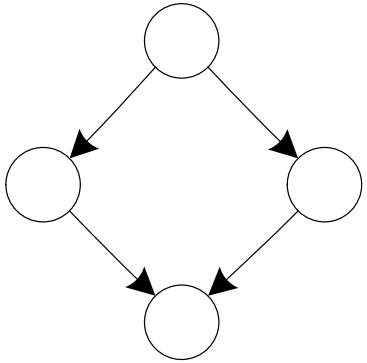
Post-test Loop



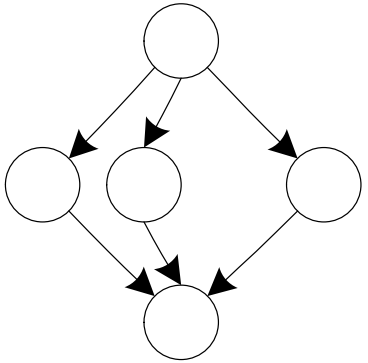
If-Then



If-Then-Else

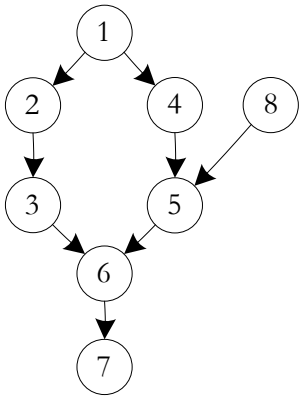


Case/Switch

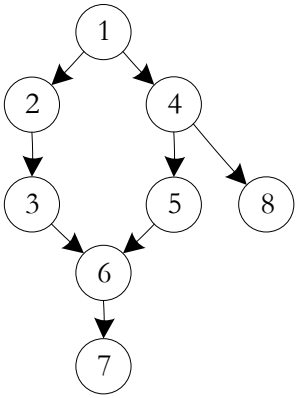


Program Graphs of Violations of the "Structured Programming" Constructs

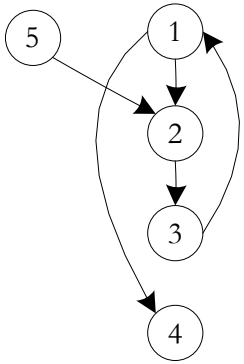
Branching into a decision



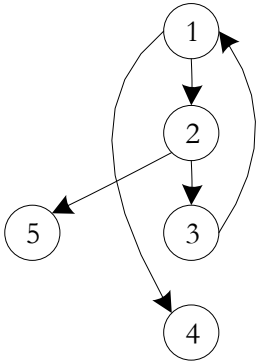
Branching out of a decision



Branching into a loop



Branching out of a loop

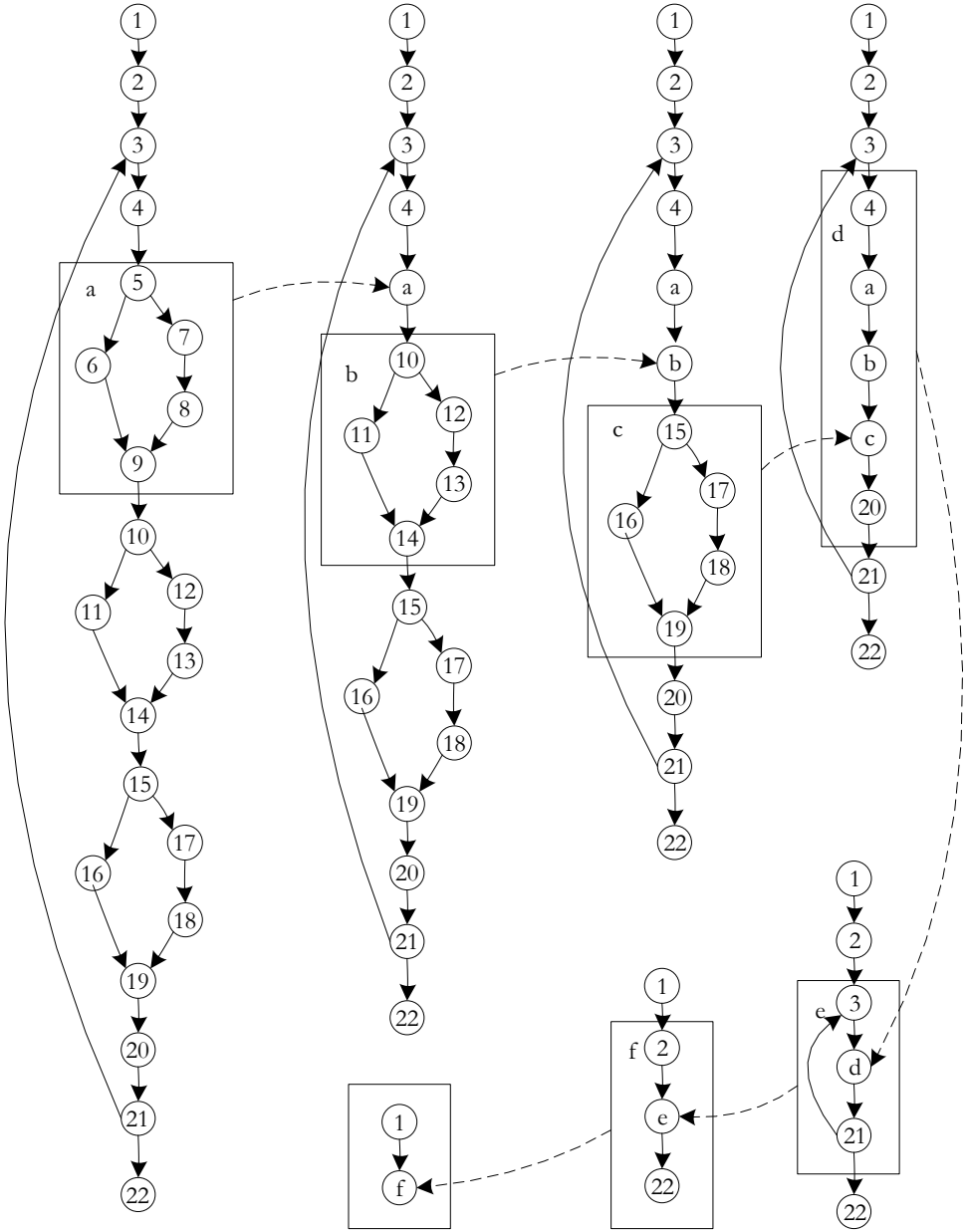


Essential Complexity

- McCabe's notion of Essential Complexity deals with the extent to which a program violates the precepts of Structured Programming.
- To find Essential Complexity of a program graph,
  - Identify a group of source statements that corresponds to one of the basic Structured Programming constructs.
  - Condense that group of statements into a separate node (with a new name)
  - Continue until no more Structured Programming constructs can be found.
  - The Essential Complexityof the original program is the cyclomatic complexity of the resulting program graph.
- The essential complexity of a Structured Program is 1.
- Violations of the precepts of Structured Programming increase the essential complexity.

Essential Complexity Example

- The next slide shows the steps to find Essential Complexity in the program graph of Figure 8.8.
- A box around a Structured Programming structure is given a lower case alphabetic name.
- In the next iteration, the box is replaced by a node with the same letter name.
- If a simple sequence is reached, the essential complexity is 1.
- If a simple sequence cannot reached, the essential complexity is the cyclomatic complexity of the remaining structure.

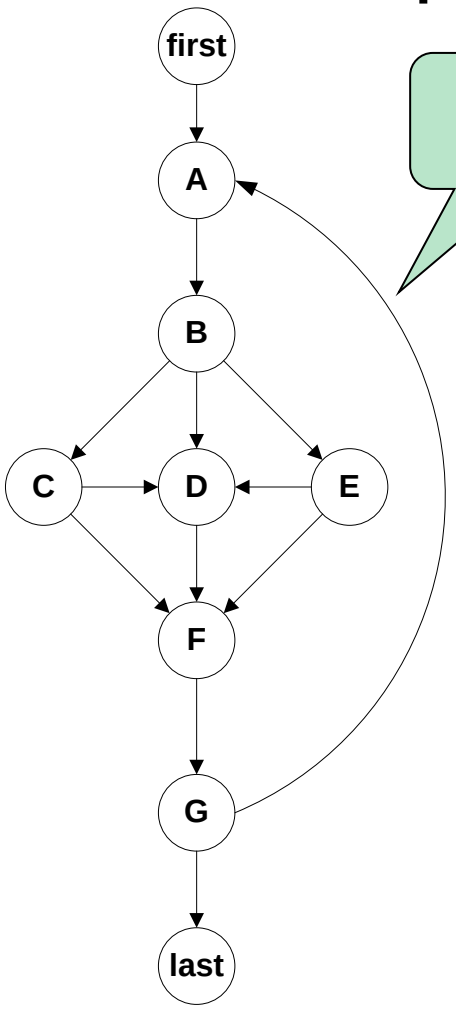

a
b
c
d
e
f

Cons and Pros of McCabe's Work

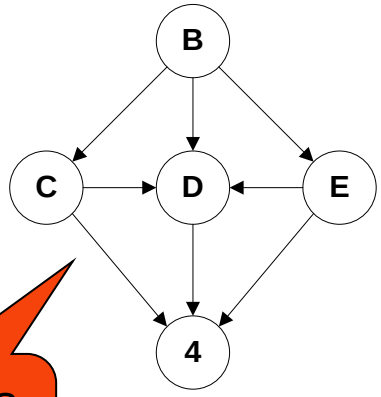
- Cons…
  - The baseline method does not guarantee feasible basis paths
  - Given a set of feasible basis paths, it may not be a sufficient test?

- Pros
  - McCabe's approach does address both gaps and redundancies.
  - Essential complexity leads to better programming practices.
  - McCabe proved that violations of the structured programming constructs increase cyclomatic complexity, and violations cannot occur singly.

Essential Complexity of Schach's Program Graph



The original



Reduces to this

V(G) = 8 – 5 + 2(1)
= 5
Essential complexity is 5

Conclusions For Code-Based Testing

- Excellent supplement (complement?) to specification-based testing because..
  - highlights gaps and redundancies
  - supports a useful range of test coverage metrics
- Test coverage metrics help manage the testing process.
- Tool support is widely available.
- Not much help for identifying test cases.
- Satisfaction of a test coverage metric does not guarantee the absence of faults.